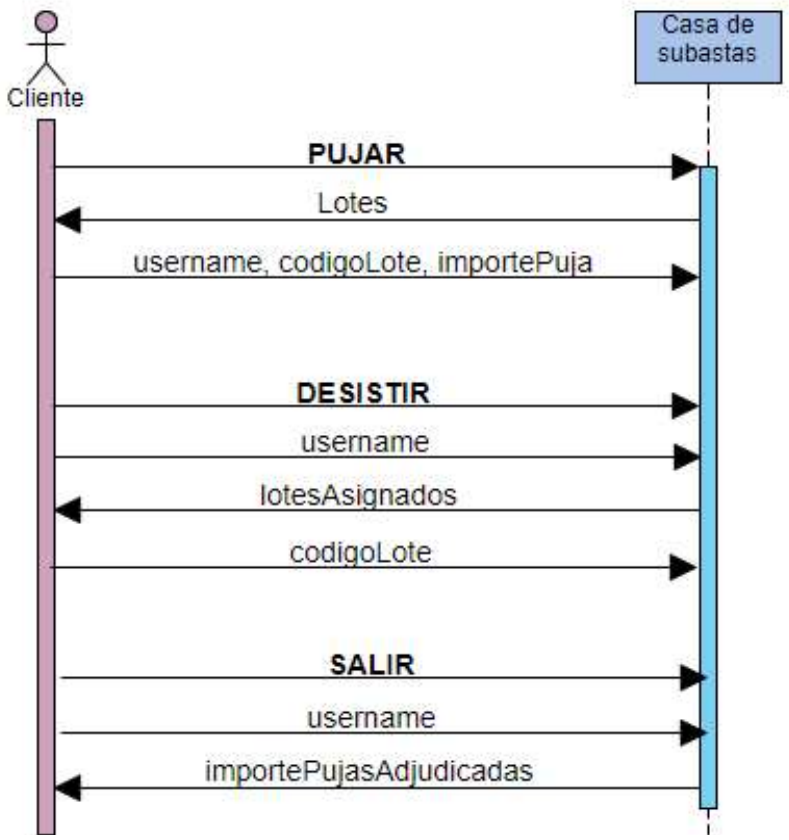
- Ejercicio3 – Subastas (4 puntos / 5 puntos)

  Queremos hacer una aplicación C/S que gestione una subasta de productos. La aplicación mantendrá un conjunto de lotes que están en proceso de subasta. La aplicación controlará las pujas que realizamos, guardando esta información junto con los lotes.

INSTRUCCIONES

- Abre los proyectos SubastasServidor y SubastasCliente
- En el **cliente no hay que cambiar nada**. Se usa sólo para probar el servidor.
- En el servidor
  - Clase Lote: **No hay que cambiar nada**
  - Clase SubastasServidor: Completad el código marcado con **TO-DO**
  - Añade todo el código necesario para crear un **servidor multihilo** que cumpla las especificaciones del siguiente protocolo:



****Todos los mensajes se pueden procesar en cualquier momento (sin estados)**

DESCRIPCIÓN DEL PROTOCOLO

✔ **PUJAR**: Cuando el servidor reciba esta solicitud

1. Deberá devolver la información de todos los productos que estén en subasta (método **toString**) concatenándola y sin separar con saltos de línea ni espacios en blanco.
2. Esperará a recibir, en un único mensaje, el nombre del usuario que realiza la puja, el código del lote por el que puja y el importe por el que puja.
   - Si el lote no existe, el servidor mostrará un mensaje de error.
   - Si el importe de la puja es menor que el importe de reserva, se mostrará un mensaje indicándolo.
   - Si el importe de la puja es menor que la mejor oferta hasta el momento, se mostrará un mensaje indicándolo.
   - En otro caso, se actualizará la información del lote con los datos recibidos.

✔ **DESISTIR**: Cuando el servidor reciba esta solicitud

1. Esperará a recibir el nombre del usuario que quiere realizar el desestimiento.
2. Deberá devolver la información de todos los productos que el usuario tenga adjudicados (método **toString**) concatenándola y sin separar con saltos de línea ni espacios en blanco.
3. Esperará a recibir, el código del lote del que quiere cancelar su puja
   - Si el lote no existe, el servidor mostrará un mensaje de error.
   - Si el lote no está adjudicado al usuario, , se mostrará un mensaje indicándolo.
   - En otro caso, se actualizará la información del lote eliminado los datos de adjudicatario y mejor oferta (ver valores por defecto en el constructor).

✔ **SALIR**: Cuando el servidor reciba esta solicitud

1. Esperará a recibir el nombre del usuario que quiere realizar el desestimiento.
2. Deberá devolver la suma de los importes de todos los lotes que el usuario tenga adjudicados. El cliente finaliza su ejecución tras mostrar el importe total. El servidor también debe dejar de atender a ese cliente.

El servidor deberá mostrar por consola los mensajes que va recibiendo y los posibles errores **según el formato que se detalla a continuación**. El cliente tiene su propia salida que no se muestra aquí. (Ver ejemplos).

EJEMPLO 1 (usuario vicente) no existente

```
Escuchando: ServerSocket[addr=0.0.0.0/0.0.0.0,localport=4444]
--> Cliente: PUJAR
La puja recibida para el lote (1) no supera el precio de reserva.
--> Cliente: PUJAR
La puja recibida para el lote (1) por el usuario Vicente es aceptada.
--> Cliente: SALIR
```

EJEMPLO 2 (usuario Luis) usuario existente

```
Escuchando: ServerSocket[addr=0.0.0.0/0.0.0.0,localport=4444]
--> Cliente: DESISTIR
El código (12) recibido no es un códgio de lote válido.
--> Cliente: DESISTIR
El lote (5) no está adjudicado al usuario Luis
--> Cliente: DESISTIR
El lote (4) se desadjudica al usuario Luis
--> Cliente: SALIR
```

EJEMPLO 3 (usuario Pedro) usuario existente

```
Escuchando: ServerSocket[addr=0.0.0.0/0.0.0.0,localport=4444]
--> Cliente: PUJAR
La puja recibida para el lote (3) no supera a la mejor puja hasta el momento.
--> Cliente: PUJAR
La puja recibida para el lote (2) no supera el precio de reserva.
--> Cliente: PUJAR
La puja recibida para el lote (3) por el usuario Pedro es aceptada.
--> Cliente: PUJAR
La puja recibida para el lote (2) por el usuario Pedro es aceptada.
--> Cliente: SALIR
```

****Hay datos precargados de usuarios y subastas para realizar las pruebas**